

Project Report
on
TRUSTBOX
-Verified Anonymous Reporting System-

B. Tech CSE- Data Science
Department of
Computer Science & Engineering
School of Engineering and Technology
K.R. Mangalam University, Gurugram-122003

Under Supervision of

Internal : Dr. Appurva Jain

External :

Submitted By-

Hritunjaya Chauhan - 2401420024

Kashish Sharma - 2401420039

Jahanvi Deswal - 2401420026

TrustBox – Verified Anonymous Reporting System

TrustBox is more than just a complaint box — it's a digital safety net designed with empathy and security in mind. It tackles the real-world issue of student silence due to fear, and replaces it with empowered, verified, anonymous voices.

Team Contributions

- **Hritunjaya Chauhan (HTML Developer):**
Built the skeleton of the site. Every form, input field, and section was structured using HTML, ensuring a clean, semantic layout for users and admins.
- **Jahanvi Deswal (JavaScript Developer):**
Powered the brain of the system. She handled OTP generation, anonymous ID creation, data flow between sections, and how everything responds to user interaction.
- **Kashish Sharma (CSS/UI Designer):**
Gave the project its visual soul. From modern dark mode themes to responsive layouts, animations, and neumorphic touches—she made the UI both beautiful and user-friendly.

Overview

TrustBox is a privacy-focused complaint submission system. It enables:

- Students to report sensitive issues (like bullying, harassment, academic stress).
- Without revealing their identity.
- But still maintaining genuine accountability via email and OTP verification.

This encourages real students to report real problems—while stopping spammers or fake reports.

Objectives

- 1. Safe and Anonymous Reporting** – Students should be allowed to make reports confidentially, knowing they will not be exposed.
- 2. Verified Entry via Confidential Unique ID** – It provides each student a unique confidential ID for verification purposes so that no false complaints can be made while protecting their anonymity.
- 3. Efficient and Secure Complaint Handling** – Protective data encryption, as well as the availability of an admin panel where authorized personnel will solve complaints responsibly.

Problem Statement & Solution

➤ **Real-World Problem TrustBox is Solving:**

Many students in educational institutions face issues such as harassment, academic pressure, infrastructure problems, or administrative neglect — but hesitate to report them. The core reason is fear of retaliation, exposure, or not being taken seriously. As a result, genuine complaints often go unreported, leaving problems unresolved and affecting student well-being.

➤ **Why This Problem is Important & How**

TrustBox Helps:

This lack of safe reporting harms both students and the institution's reputation. Without feedback, schools cannot improve or protect students effectively.

TrustBox solves this by providing a secure, anonymous complaint submission system:

1. Students verify through OTP (to prove they're genuine).
2. Their identity remains hidden.
3. The system flags repeated complaints for priority action.

This encourages students to speak up safely, helping institutions to identify and solve critical issues early.

Methodology

The TrustBox system follows a structured development process to address anonymous student reporting through secure authentication and complaint handling. The workflow includes:

- ☐ User Enters College Email
- ☐ Simulated OTP Sent & Verified
- ☐ System assigns Anonymous ID
- ☐ User writes complaint
- ☐ Complaint saved to storage
- ☐ Admin reviews flagged reports

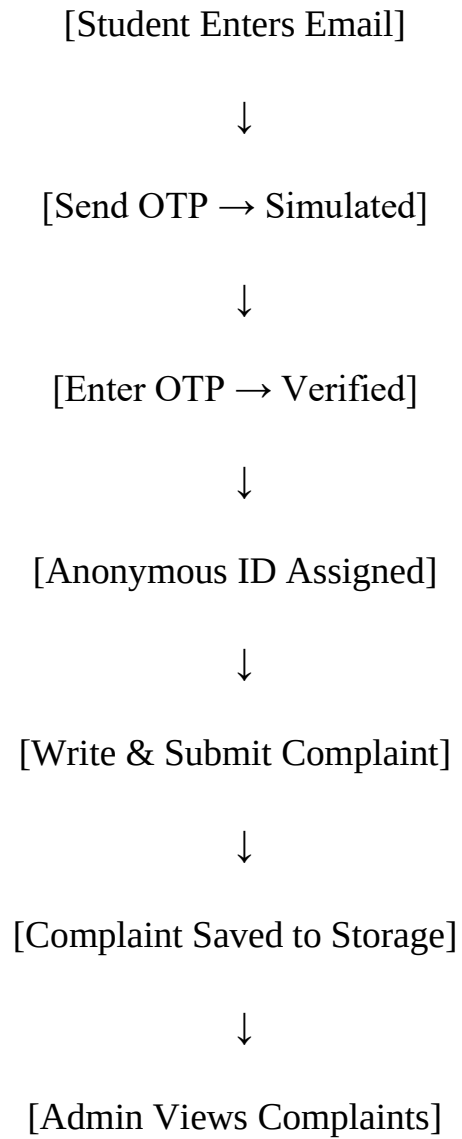
This simple flow was structured to be:

- User-friendly
- Admin-manageable
- Secure and scalable

System Architecture

Layer	Technology Used	Description
Frontend	JavaScript	User interface for students and admins
OTP Engine	JavaScript (Simulated)	Sends and verifies OTP for user validation
Anonymous Layer	JavaScript Logic	Assigns unique Anonymous IDs after OTP verification
Storage	localStorage (or MongoDB in full version)	Temporarily stores complaints
Admin Panel	JavaScript + DOM	Allows viewing submitted complaints

FLOWCHART



Functional Approach

A. OTP-Based Verification

- User inputs college email.
- OTP is generated on the frontend (simulated for demo).
- OTP is verified; upon success, user receives a random anonymous ID.

B. Complaint Submission

- Verified users can write and submit complaints.
- Complaints are stored with an anonymous ID in localStorage (can be replaced with MongoDB later).
- Each complaint includes:
 - Anonymous ID
 - Complaint Text
 - Timestamp (optional)

C. Admin Interface

- Admin logs in using a predefined password.
- All submitted complaints are retrieved and displayed.
- Interface includes visual blocks for each complaint.
- Can be extended to include:
 - Filtering by category
 - Marking status (Pending/In Progress/Resolved)

TOOLS AND TECHNOLOGIES USED

Technologies Used:

- HTML:

Defines the structure and semantic layout (buttons, inputs, divs).

- CSS:

Embedded in the <style> tag to style elements with gradients, animations, shadows, and rounded corners.

- JavaScript :

Handles UI logic, OTP generation, form transitions, and local data storage using localStorage.

Front-end

Layout & Structure:

- Single Page Interface:
- Uses hidden
- sections like #userSection, #adminSection, etc., shown/hidden dynamically using JavaScript.
- Buttons and Inputs:
- Buttons trigger different flows (login, submit complaint, view complaints).
- Input fields for OTP, email, password, and complaints.

Styling Details:

- Dark Mode Theme:
Dominant colors: deep maroon, black, gray.
Visually calming, privacy-friendly tone fitting an anonymous system.
- Neumorphism Elements:
Soft shadows and gradients create depth on input boxes and buttons.
- Responsive Design:
Container width set with percentages and max-width.
Central alignment with Flexbox ensures adaptability across devices.

Functional Logic:

- OTP Simulation:
Random 6-digit OTP is generated and shown via alert() (not sent via email).
- Admin Authentication:
Simple password check (admin123) for demo purposes.
- Local Storage:
Complaints are saved and retrieved using localStorage.

UX/UI DESIGN

Wireframe-Like Structure:

- Home View: Choose "User" or "Admin" login.

User View:

- Enter email → send OTP
- Enter OTP → login anonymously
- Submit complaint

Admin View:

- Enter password
- View complaints with status tags
- User Experience (UX) Principles Applied:
- Simplicity: Clear flow for each role (User/Admin), minimal navigation.

Feedback:

- Alerts for wrong OTP, invalid email, submission confirmation.
- Hover animations provide tactile feedback on buttons.

Status Indicators:

- Each complaint is marked as:
- Normal
- Priority (same complaint from multiple users)
- Suspicious (same user submitted same complaint multiple times)

Tools:

- While not used here, for actual wireframing and prototyping, designers might use:
- Figma
- Adobe XD
- Sketch

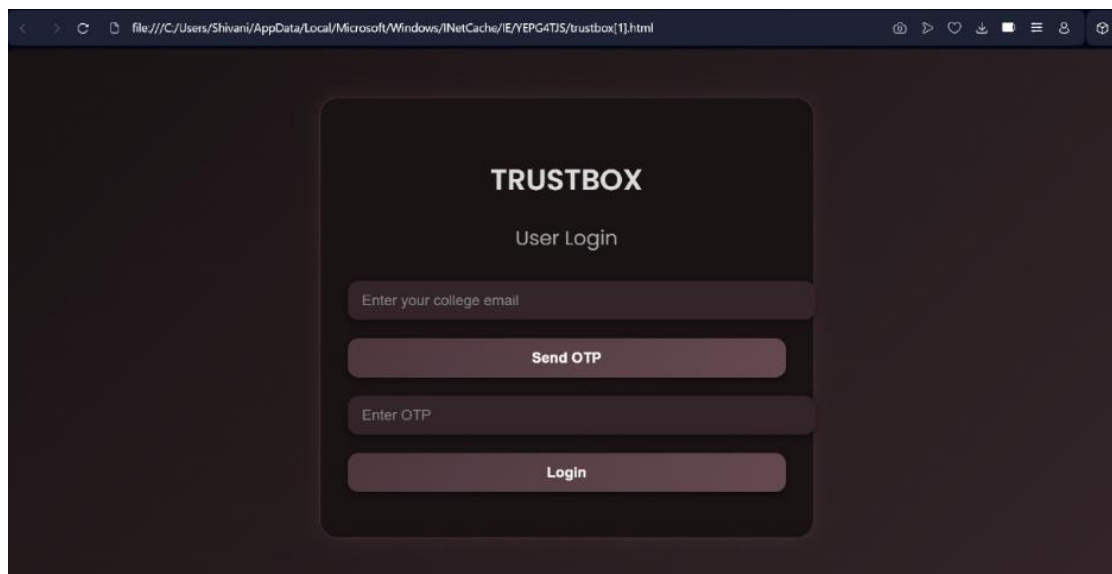
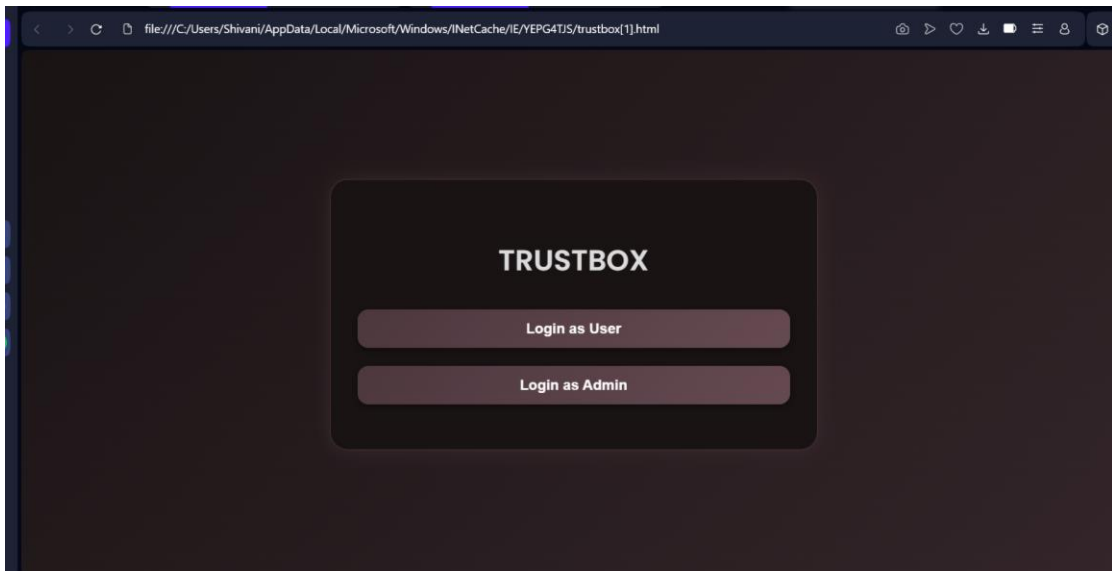
Design Rationale:

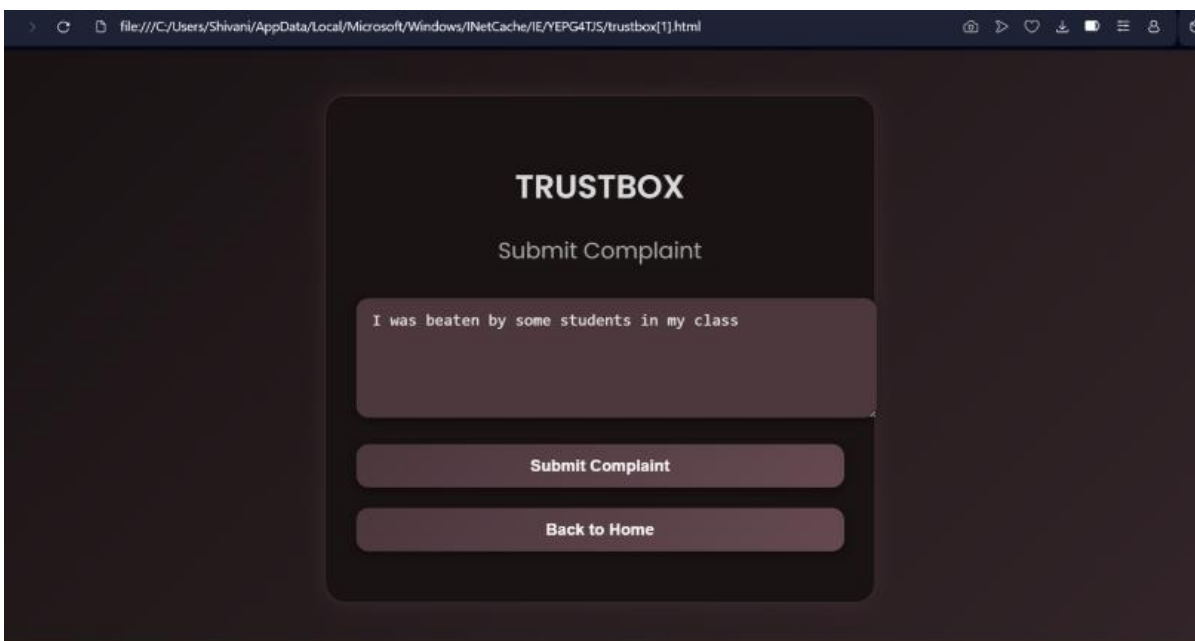
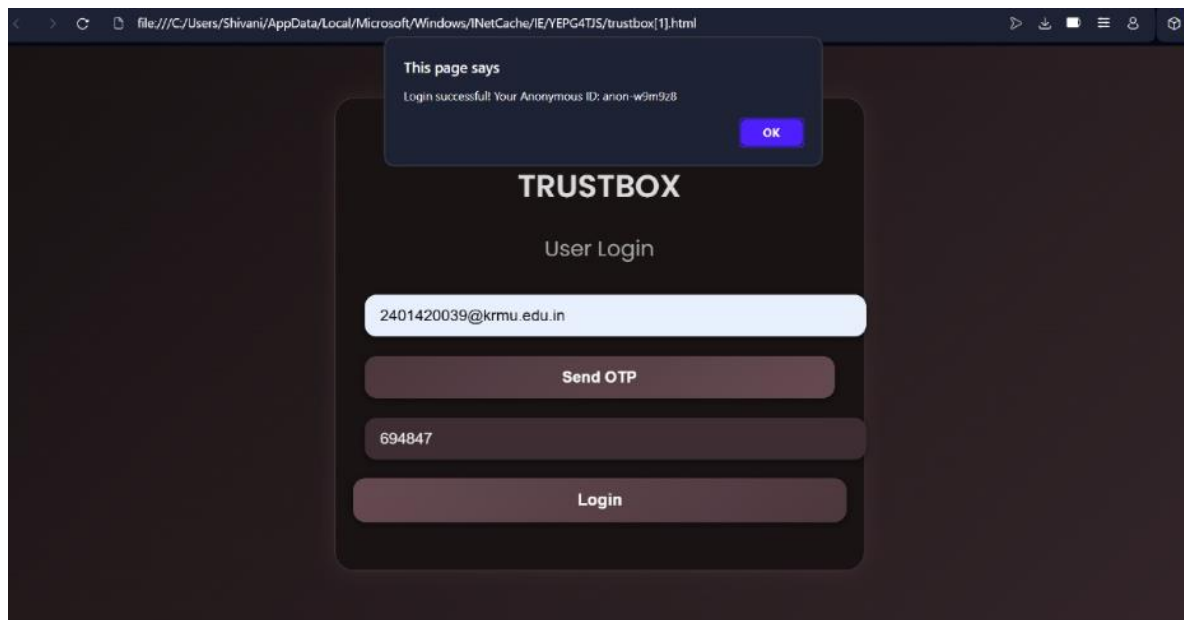
- Encourage anonymous, honest reporting by:
- Not requiring passwords for users.
- Providing feedback after submission.
- Using a clean, non-intimidating interface.
- Admin view focuses on pattern detection in complaints using JavaScript analysis.

Demonstration

USER LOGIN:

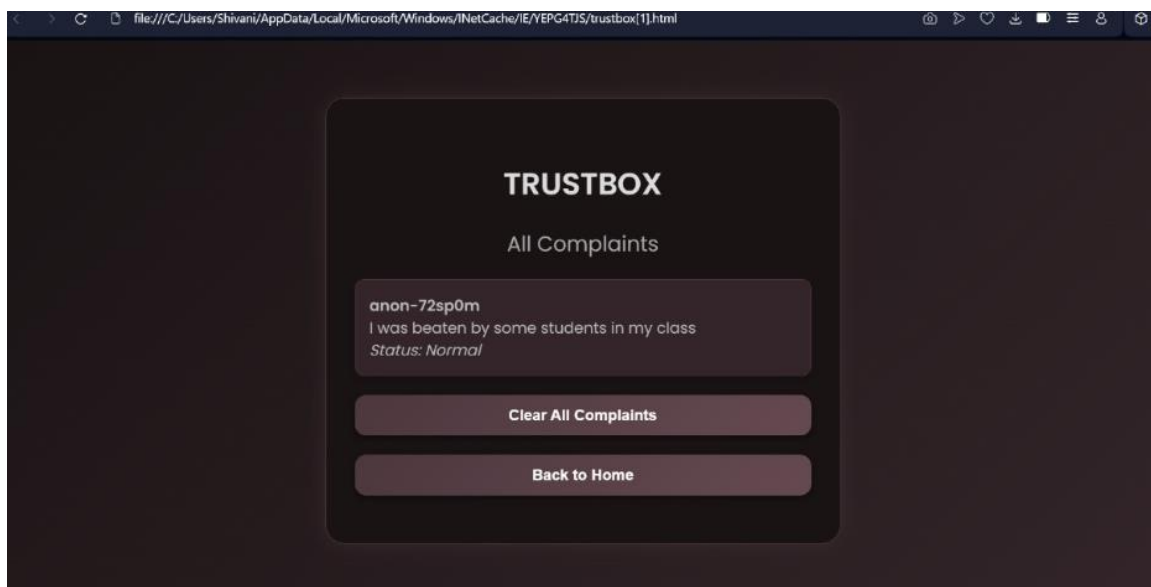
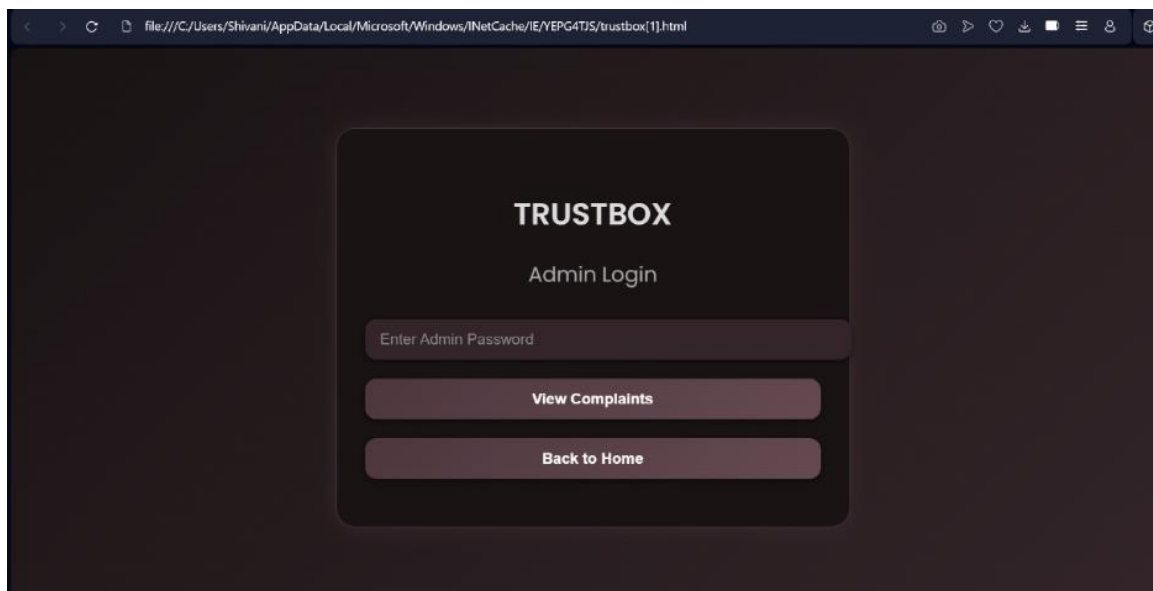
- 1) Enters email (@krmu.edu.in)
- 2) Clicks Send OTP
- 3) Enters OTP → receives Anonymous ID
- 4) Submits complaint → directed to home screen





ADMIN:

- 1) Enters password
- 2) Views complaint dashboard
- 3) Flags, reads, and optionally clears complaints.



Key Features

- Anonymous login with college email (only @krmu.edu.in is allowed)
- OTP-based verification (simulated)
- Auto-generated anonymous IDs
- Complaint submission form
- Admin view with complaint analysis
- Priority: Same complaint reported by multiple users
- Suspicious: Same complaint reported multiple times by one user
- Ability to clear all complaints (admin only)
- Responsive UI with animations and secure interaction prompts

Conclusion:

Learning from the project

Project Reflection

Learning and Experience:

Through this project, we gained valuable hands-on experience in building a complete web application from scratch.

We learned how to:

- Design a clean, responsive frontend using HTML, CSS, and animations to enhance user experience.
- Implement client-side functionality with JavaScript, including email validation, OTP generation, anonymous user IDs, complaint submissions, and admin authentication.
- Manage data storage using localStorage while understanding its limitations in terms of security and scalability.
- Collaborate effectively by dividing roles — focusing on frontend design, application logic.— which gave us a real-world feel of teamwork in a software development project.
- Identify security challenges and explore ways to make data handling more secure.
- This project helped strengthen our skills in problem-solving, coding best practices, and user-centric design thinking

Future Enhancements

We have identified several areas where we can improve and extend the application:

Implement a Real Backend:

Move from localStorage to a secure backend database like Firebase or MongoDB with an Express.js server.

Email-based OTP Delivery:

Integrate a real email service (e.g., SendGrid, EmailJS) to send OTPs securely instead of simulating them.

Enhanced Admin Dashboard:

Create a full dashboard for the admin with filtering options, complaint categories, and analytics (such as number of complaints per week).

User Session Management:

Add session timeouts and secure login/logout functionality.

Mobile Responsiveness:

Improve UI for small devices and add offline capabilities using Service Workers.

Complaint Status Tracking:

Allow admins to update the status of complaints (Pending, In Review, Resolved).

Multi-Admin Support:

Enable multiple admin accounts with role-based access control.

Thank You